Design & Develop a Personal Finance Tracker Web Application

Objective:

Design and develop a **Personal Finance Tracker** web application that allows users to track their income, expenses, and savings. The application should have user authentication and provide visualizations of spending, categorization of expenses, and monthly budget summaries. The goal is to create an intuitive and visually appealing app to help users manage their finances effectively.

Part 1: Design (Figma)

1. User Flow & Requirements:

- Users will have the ability to:
    - **Sign up** and **log in** to the application (authentication).
    - **Add income** and **add expenses** to their financial records.
    - **View spending history** by category (e.g., Food, Transport, Entertainment, etc.).
    - **Set and track a budget** for each category.
    - **Visualize financial data** (e.g., pie charts, bar graphs).
    - **Edit** or **delete** financial entries.
    - View an **overview** of their finances (balance, monthly expenses, savings, etc.).

2. Wireframing the Pages:

- **Dashboard (Home Page):** Displays a financial summary (total balance, monthly income/expenses), a chart of spending by category, and a list of recent transactions.
- **Add Transaction Page:** A form to add a new income or expense (with fields for description, amount, category, date).
- **Expense History Page:** Displays a list of past transactions with options to filter by date range or category.
- **Budget Page:** Allows users to set monthly budget limits for each category (e.g., set $200 for food, $100 for entertainment).
- **Login/Register Pages:** For user authentication.

3. High-Fidelity UI Design:

- Convert your wireframes into high-fidelity designs in **Figma**.
- Use **charts and graphs** to visualize financial data (pie charts, bar charts, etc.).
- Design **input forms** for adding transactions, ensuring ease of use (clear labels, dropdowns for categories).
- Focus on **clean, minimal design** that is easy to navigate.
- Ensure the design is **responsive**, with mobile and tablet views.

Deliverables:

- Figma files with wireframes, high-fidelity UI designs, and user flows.
- Style guide for visual consistency across the app (buttons, charts, form fields).

Part 2: Front-End Implementation (in .NET MVC)

1. Set Up .NET MVC Application:

- Create a new **.NET MVC application** that will handle user authentication and display data (e.g., user transactions, budgets).
- Use **Razor Views** for rendering pages dynamically based on user data.
- Set up models for users, transactions, and budgets (using **Entity Framework** to interact with the database).

2. Pages to Implement:

- **Dashboard (Home Page):**
    - Display a financial summary with total balance, income, and expenses for the month.
    - Use **Chart.js** or another library to create visualizations of spending by category.
    - Include a list of recent transactions with options to edit or delete.
- **Add Transaction Page:**
    - A form to allow users to add income or expense records.
    - Use **AJAX** to submit forms without refreshing the page (so users can instantly see the updated data).

- **Expense History Page:**
    - Display a table of past transactions, sortable by date or category.
    - Provide filters by date range or category (e.g., show only "Food" expenses for the past month).
- **Budget Page:**
    - Allow users to set a budget for each category (e.g., $300 for groceries, $100 for entertainment).
    - Provide a visual indicator showing how much of the budget has been spent (progress bar or pie chart).
- **Login/Register Pages:**
    - Implement basic user authentication (sign up, log in, and log out).
    - Store user session data and ensure users only see their own transactions.

3. Implement Financial Data Management:

- **CRUD Operations:**
    - Implement **Create, Read, Update, Delete** operations for financial transactions.
    - Allow users to update or delete income/expense entries.
- **Categories for Transactions:**
    - Create predefined categories for income and expenses (e.g., Rent, Food, Salary, etc.).
    - Allow users to assign a category to each transaction.

4. Charts & Visualizations:

- Use **Chart.js** (or similar JS library) to render:
    - **Pie charts** to visualize spending by category.
    - **Bar charts** for income vs. expenses over time.
    - **Progress bars** for budget tracking.

5. User Authentication:

- Implement user authentication with **ASP.NET Identity** or simple session-based login.
- Ensure users can sign up, log in, and view only their own financial data.

6. Responsive Design:

- Ensure the application is **responsive**, with clean layouts on desktop, tablet, and mobile.
- Use **CSS Grid** or **Flexbox** for flexible layouts.

7. Testing:

- Test **cross-browser compatibility** to ensure the app works across all major browsers (Chrome, Firefox, Safari, Edge).
- Test the **responsive design** to ensure the app displays correctly on all screen sizes.
- Verify the **CRUD operations** work properly, and data is updated dynamically.
- Test that **charts and graphs** render correctly with the data.

Deliverables:

1. **Design Deliverables:**
    - Figma files with wireframes, high-fidelity UI designs, and user flows(a plus).
    - Style guide for visual components (buttons, input fields, chart styles).
2. **Development Deliverables:**
    - Fully functional **.NET MVC application** with the following:
        - Dashboard (financial summary and charts).
        - Add/Edit/Delete transactions.
        - View past transactions and history.
        - Set and track budgets by category.
        - User authentication (login/register).
    - **Responsive design** that works on mobile, tablet, and desktop.